

# A Cache-enabled NDN Forwarding Plane based on Programmable Switches

Lei Yu<sup>1</sup>, Shuai Gao<sup>1,\*</sup>, Ningchun Liu<sup>1</sup>, Hongchao Wang<sup>1</sup>, Wei Su<sup>1</sup>

<sup>1</sup>School of Electronic and Information Engineering, Beijing Jiaotong University, Beijing, China

\*Corresponding author

Email: {lei.yu, shgao, nchliu, hcwang, wsu}@bjtu.edu.cn

**Abstract**—Named Data Networking (NDN) is a promising network architecture proposed to realize efficient information distribution. Unlike TCP/IP, NDN uses content names to forward packets. As a result, the deployment of NDN on current network equipment faces challenges. Meanwhile, the rapid development of the programmable data plane makes it possible to implement NDN with Programmable Data Plane (PDP) switches. In this paper, we propose a cache-enabled NDN forwarding plane using P4 in which massive caching and file transfers are well supported. A dedicated process is designed to take over the cache and communicate with the P4 switch process. Hash indexes are used in PIT and CS to improve query speed. In addition, we improve the NDN multicast pipeline design to support efficient and reliable file transfer services. Experimental results show that our NDN switch can achieve considerable throughput improvement.

**Index Terms**—NDN, programmable data plane, cache

## I. INTRODUCTION

With the rapid development of network technologies, content service represented by online video has gradually become the mainstream service of the Internet. The traditional TCP/IP network adopts a host-centric communication mode, facing many challenges such as redundant transmission. Since 1999, a series of ICN architectures have been proposed. ICN adopts an information-centric communication mode and decouples the location of content and content providers. In this case, content copies can be cached at intermediate routing nodes to respond to the exact subsequent requests. Due to this reason, caches can be adopted in the network to reduce a redundant transmission of a content copy.

As the most promising project in ICN architecture, NDN [1] follows the content-centric design concept and addresses and routes based on named data in the network layer. Because the forwarding based on named data in NDN is essentially different from the packet processing logic of traditional TCP/IP networks, and the implementation of existing NDN routers is mostly software-based. This greatly limits the forwarding performance, making NDN encounter challenges in large-scale deployments. The emergence of the Programmable Data Plane (PDP) provides a potential for the construction of high-performance NDN routers and the large-scale deployment of NDN [2], [3].

P4 programmable switch is composed of programmable ASIC, FPGA, NPU, or CPU. Building on this, P4 language focuses on protocol-independent features in design but lacks

support for memory allocation. In order for the programmable switch to manage a larger memory space for NDN caching on the data plane, several possible solutions are a) Using the P4 switch register unit to realize the cache [4]. It can only support small-capacity data fragments. b) By modifying the BMv2 source code of the software switch, the Data cache can be realized by adding a cache queue [5] or a cache unit. The disadvantage is that the existing P4 pair external module support is limited. c) Non-volatile storage attached to the device is invoked through the external interface. But this results in longer packet delays and additional mapping.

In this paper, to address the above challenges, we design a cache-enabled NDN forwarding plane based on programmable switches. The simplified NFD communicates with the P4 switch process through the Face built by the local virtual Ethernet port, which provides low-latency access to the cache. The cache management process does not block the normal forwarding of the switch. It can be deployed locally (low latency, takes up local memory) or on additional devices (higher latency, independent resource usage).

Our main contributions can be summarized as follows:

- (1) We design a cache name list the associated package processing flow in the P4 switch to reduce the query delay.
- (2) We implement a PIT-guided packet-clone multicast in egress to reduce the blocking time of Data processing by using P4 multithreading.
- (3) We extend the NDN data plane with the Interest retransmission mechanism of NDN switches to support reliable file transfer services using NFD.

The rest of this paper is organized as follows. Section II surveys the related work. In Section III, we describe the NDN data plane design. Section IV illustrates the capabilities of this design with a prototype implementation. Finally, section V concludes this paper.

## II. RELATED WORK

This section reviews related work on NDN switch design and P4 cache design.

### A. NDN switch design

A mature software-based design is NDN Forwarding Daemon [6]. NFD is an open-source NDN software switch provided by the NDN project. NFD stores cache Data in

memory in the form of a full-name sorting list. To facilitate the deployment of cache strategy, NFD also defines some additional attributes, such as when the data become non-fresh.

As a first attempt at implementing an NDN router in P4, NDN.p4 [7] implements many features of NDN logic. However, due to the constraints of state storage in P4-14, the implementation cannot cache requests. Miguel et al. [8] used the new function of P4 to expand NDN.p4 and optimize its operation through the request caching mechanism. The caching mechanism is implemented with P4 externs, but P4 target BMv2-ss cannot support this function. Enhanced NDN (ENDN) [9] is an advanced NDN architecture. It provides a larger content delivery function directory, such as adaptive forwarding, custom monitoring, network cache control, and publish/subscribe forwarding. Extern functions are used to access the string-based NDN packet content names. Guo et al. [10] proposed an NDN routing mechanism Combining the NLSR protocol and the P4 environment. The NDN routing services are provided by the control plane application using resource-location mapping to achieve part of the NLSR mechanism.

Takemasa J et al. [11] validates that the architecture is able to achieve the 10-Tbps throughput of Data by analyzing bottlenecks for high-speed forwarding. The prototype implementation with a Tofino switch and a commodity server demonstrates about 470 Gbps throughput when using a single pipeline of the switch ASIC. As a comparison, in 2013, Cisco [12] implemented an NDN data plane with a software forwarding engine on an Intel Xeon-based line card in the Cisco ASR 9000 router. The NDN router can forward the NDN traffic at 20Gbps or higher. As main take-away, the throughput of these software-based solutions is still orders of magnitude below what the new generation programmable switches can sustain.

### B. P4 cache design

There are some cache implementations on P4 switches for other use cases. Jin et al. [4] use the software switch register unit to achieve cache, which has the best performance on P4. The size of the register unit of the software switch is limited. It is impractical to support packet buffering in MTU 1500 scenarios without modifying the source code. At the same time, other match action tables also use register units to store and match related table entries. Misuse of registers can lead to performance degradation of the switch. Qu et al. [5] proposed a P4 queue cache. Queue caching refers to the use of a virtual port to store the datagram, retrieve the datagram when it leaves the queue, and then let the datagram enter the queue again to store the datagram for Packet Loss Recovery. The cache capacity required by NDN is larger than the capacity provided by the above solutions, so it is necessary to find another way.

## III. CACHE-ENABLED NDN FORWARDING PLANE DESIGN

The challenge in NDN switches design on P4 is that NDN packets use nested TLV structures to organize variable length fields in packet parsing, requiring stateful storage on switches

and random multicasting in forwarding. In the above aspects, substantial improvements have been made to enable reliable file transfer services based on the NDN.p4 scheme. This section introduces the data plane design in detail from the following three parts: data plane structure overview, packet parsing module, pipeline, and cache management module.

### A. Overview

P4 programming structure mainly includes four modules: parser, ingress pipeline, egress pipeline, deparser. PIT and CS are stored in the data plane and sent by the switch, while the FIB and Ethernet address is configured by the control plane. NDN uses faces to abstract the transmission interface. A face can be a connection with other network nodes or connection with local applications through inter-process communication. Communication between the BMv2 switch and the cache server over the local face uses NDN native packets. In this design, the cache management process is responsible for caching and responding to packets. BMv2 switches are connected to other network devices through physical Ethernet cards.

The cache module needs to realize the sending and receiving, storage, query matching, and cache strategy deployment. The cache module is separated from the forwarding module. We maintain the cache name list in the P4 switch based on the hash index to reduce the query delay, and the corresponding packets will be stored by the cache management process rather than by the P4 switch. In this scheme, the Interest is forwarded to the cache management process after the cache hit, and the corresponding Data is returned. When the switch needs to access the cache, it only needs to query the name hash without integrating the entire cache module into the P4 program. Separating forwarding and cache management improves the performance of large-capacity cache based on P4 forwarding.

### B. Parser Module

The parser specifies the length and sequence of parsing header fields by a programmable switch. The parser of the P4 switch is a finite state machine. After a parser fragment is carried out, the packet will enter the next parser fragment according to the order of the parsed results, that is, to complete the transfer between the two states. Because the NDN content name is composed of multiple continuous components TLV, we use the Header Stack data structure to simplify this series of the same type of adjacent header parsing. For continuous component TLV, P4 will repeat the TLV header parsing process several times until all component parsing is completed. We also expanded the header parsing process to support version-discovery of the NDN protocol. All three types of packets must pass through the name parser: Interest(0x05), Data(0x06), and NAK(0x0a).

### C. Ingress Pipeline & Egress Pipeline

Firstly, the variable name is converted into a fixed-length hash string by the hash function to be stored, parsed, and matched. FIB and PIT of NDN match the routing Interest by

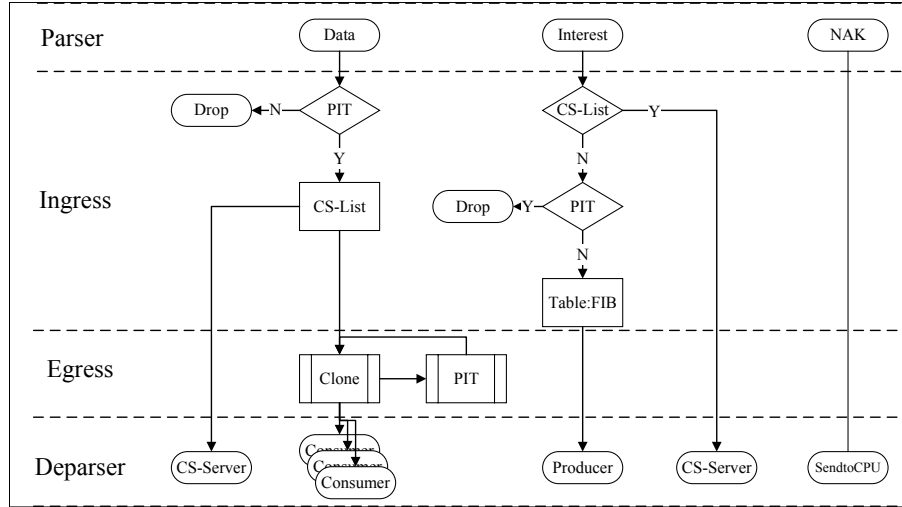


Fig. 1. Pipeline flow chart.

the longest prefix of the content name. All packets must be converted from name to hash to enter the forwarding process. Hash is stored in P4 custom metadata, which is only used for internal processing of the switch. Hash index and PIT table stored in CS will not be sent at the port and will be released after the packet processing process.

PIT and CS-table are stored in the register and handled by the NDN switch itself, while the FIB and Ethernet address is configured by the control plane. Stateful forwarding minimizes the unnecessary communication between the control plane and the data plane to meet the decoupling requirements of the control plane and the data plane.

Then, as shown in Fig.1, different processing is performed depending on the type of the packet:

1) *NAK*: When receiving an *NAK* packet, the programmable switch will forward it to the controller in the control plane for processing.

2) *Interest*: When receiving an *Interest*, the programmable switch will process it as follows.

**Step 1:** Match in the CS-list to determine whether there is a cache. The *Interest* is directly forwarded to the cs port when data is left in the cache.

**Step 2:** Enter the PIT (recording the previously forwarded *Interest* and their faces entering the switch) for matching. If the corresponding entry is found in the PIT, the routing node has forwarded the same requested *Interest* but has no response yet. It is only necessary to discard the *Interest* and update the face to the PIT. If the *Interest* is received multiple times on the same face, it is regarded as a retransmitted packet, so the matching action table FIB is applied.

**Step 3:** If the corresponding matching table entry is not found in the PIT, then the content request is processed by the routing node for the first time and needs to be searched in the FIB (recording the forwarding rules between different nodes). The request is forwarded to other routing nodes according to the interface list corresponding to the content name in the

FIB, and the PIT entry corresponding to the content name is modified to indicate to the next same content request that the *Interest* of the content name is being requested to be resolved.

3) *Data*: When receiving a *Data*, the programmable switch will process it as follows.

**Step 1:** Packets enter the PIT for matching. No matching entry means it is an unsolicited packet and should be dropped.

**Step 2:** If there is a match in the PIT, the name of the *Data* should be stored in the CS-list to wait for other consumer requests. The router should forward the packet to several faces and the CS-Server port in turn according to the PIT. The multicast of *Data* is implemented in the egress pipeline. First, the original packets are forwarded to the port with the lowest serial number, and then a loop is constructed through PIT-clone. Every time a *Data* is sent, the corresponding bit in the PIT is set to zero. And enter the *Data* sending process of the next port until the PIT entry value is zero. The last port defaults to the cache server port.

When the P4 switch performs the clone operation in the egress, a new thread is created to re-execute the egress process each time a new packet is cloned. During this period, the switch can continue to process the newly arrived packets to make full use of port resources. Implementing clone multicast based on PIT guidance in egress can reduce the blocking time of *Data* processing by using P4 multi-threading.

#### D. Cache Module

The cache management process accepts all *Data* for storage, manages the data according to the required cache replacement policy, and returns the corresponding data when receiving the *Interest*. The cache replacement strategy adopted by the cache management program should be the same as the cs-list update strategy in the P4 program. The cache management program is attached to the P4 switch and does not communicate directly with the outside world, nor is it affected by the control plane. To utilize existing resources as much as possible, we can build

a simple cache management program based on well-designed NFD components.

In the native NDN design, the cached query has high priority and responds quickly, but it is challenging to implement due to deployment conditions. P4 in-program processing cache can significantly degrade P4 switch performance. In the scheme of this paper, the cache function is externalized, the cache query and storage management are separated, and the main factor affecting the delay of the cache query is no longer in the processing flow of P4. In addition, we optimize parsing, multicasting, and retransmission to minimize forwarding delay and improve throughput.

#### IV. PERFORMANCE EVALUATION

In this section, the performance of the NDN cache implementation will be analyzed from two metrics: latency and throughput. The topology is shown in Fig.2.

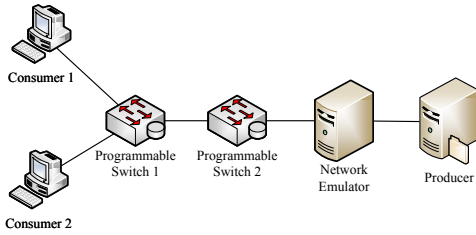


Fig. 2. The topology of the prototype system.

The scenario consists of producers, consumers, NDN switches, and link emulation devices connected by Ethernet links. Consumers continuously send different Interest, and producers provide corresponding Data to the network. The NDN router adopts a FIFO cache strategy, and the upper limit of the cache is 16GB. Producer link latency is set to a random delay from 2.5ms to 5.0ms to simulate the Internet environment (A delay variation distribution similar to the jitter observed in the wide-area network provided by Sprirent). The device parameters are summarized in Table I.

TABLE I  
THE SUMMARY OF DEVICE PARAMETER IN THE PROTOTYPE SYSTEM.

| Device              | Parameter                                   | Count |
|---------------------|---------------------------------------------|-------|
| Programmable Switch | Intel Xeon Gold 5218@2.30GHz<br>64.0 GB RAM | 2     |
| Consumer            | Intel Xeon E3-1230V2@3.30GHz<br>4.0 GB RAM  | 2     |
| Producer            | Intel Xeon E5-2620@2.00GHz<br>16.0 GB RAM   | 1     |

##### A. Content Retrieval Latency

The content retrieval latency that a consumer request a single Data is measured to investigate switch performance. We measured content retrieval latency with and without caching, giving average results over a hundred tests. As a comparison, we tested the latency performance of NFD and NDN.p4 as

TABLE II  
THE COMPARISON OF CONTENT RETRIEVAL LATENCY.

|            | From producer | From CS |
|------------|---------------|---------|
| NFD [6]    | 5.30ms        | 0.96ms  |
| This paper | 5.77ms        | 1.84ms  |
| NDN.p4 [7] | 6.35ms        | -       |

software switches in the same scenario, and the results are shown in Table II.

With the near-end cache, the NDN switch can effectively reduce the delay by about 5ms, approximately equal to the overhead for remote access. The content retrieval latency of the proposed P4 switch scheme is 0.58 ms lower than that of the NDN.p4 scheme (NDN.p4 has no cache on this target). This comes from the optimization we did on the pipeline. The forwarding speed of BMv2 (this paper) deployed on a general-purpose server is close to that of NFD, which can meet the requirements of NDN.

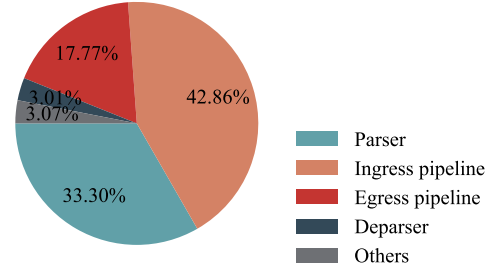


Fig. 3. Time cost of the main blocks of P4 program.

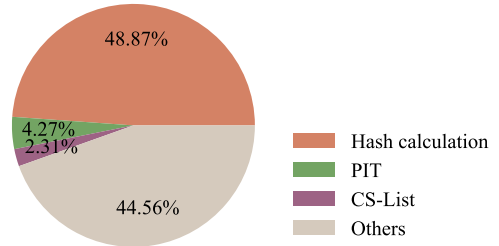


Fig. 4. Time cost of the main blocks of ingress pipeline.

The time-cost for BMv2 to process a single NDN packet is about 1ms. The BMv2 timestamp can measure the processing delay of each part of the packet forwarded by the switch. The results(Fig.3, Fig.4) show that the parser and hash calculation overheads are the most time-consuming part. The nested structure of the NDN packet makes the switch unpack it layer by layer while reading the length of each segment one by one, which increases the cost of the parser part. In ingress, the overhead of hash calculation is much larger than the overhead

of reading registers, which, together with hash calculation, accounts for half of the packet processing latency. Fortunately, packet multicast only performs the egress and deparser parts (20.78 %) without repeating the parser and ingress [7].

### B. Throughput

In this scenario, we measure the traffic of a consumer requesting a 16MB file which is about 15,000 data requests. The slow-start and congestion avoidance algorithms for congestion control are deployed. First, we measure the real-time traffic without the cache and then measure it with the near-end cache. We added an Interest retransmission mechanism for NDN switches to support NFD for reliable file transfer services.

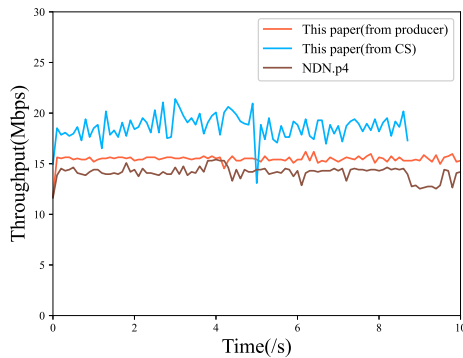


Fig. 5. Throughput of 16MB file retrieval.

Fig.5 shows that when the cache is not used, the average Throughput is about 15Mbps. When the cache is used (hit rate >99%), the average Throughput is about 19Mbps(file transfer ended in 8.82s). Under the same hardware conditions, the average Throughput of NDN.p4 is 13Mbps, which shows that our optimization scheme is effective. The traffic fluctuation at the end of the curve is related to the packets retransmitted over time. This is mainly due to the multi-threaded design of BMv2. Every time a new packet is cloned, a new thread will be created, which may lead to continuous processing of multicast tasks. Multiple clone queues in a process cause high queuing delays for individual clone packets.

### V. CONCLUSION AND FUTURE WORK

In this paper, we present the design, implementation, and performance evaluation of a cache-enabled NDN forwarding plane on P4. The switch takes the separation of cache management and packet forwarding as the framework. We designed a hash index-based cache name list and PIT-guided clone multicast in egress to reduce query latency. We extend the NDN data plane with the Interest retransmission mechanism of NDN switches to support reliable file transfer services. Our solution can achieve 20Mbps throughput on BMv2, which is higher than other P4 solutions.

Next, we will evaluate our solution on programmable networking hardware. Specifically, the NDN-P4 scheme will be tested on Intel's Tofino switches. Considering the current

performance of BMv2 and the improvement brought by Tofino hardware support, it is hoped that this solution will show higher performance.

### ACKNOWLEDGMENT

This work was supported by the National Key Research and Development Program of China (No. 2019YFB1802503), the National Nature Science Foundation of China (No. 61972026, No. 61802014).

### REFERENCES

- [1] L. Zhang, A. Afanasyev, J. Burke, V. Jacobson, K. Claffy, P. Crowley, C. Papadopoulos, L. Wang, and B. Zhang, "Named data networking," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 66–73, 2014.
- [2] E. F. Kfoury, J. Crichigno, and E. Bou-Harb, "An exhaustive survey on p4 programmable data plane switches: Taxonomy, applications, challenges, and future trends," *IEEE Access*, vol. 9, pp. 87094–87155, 2021.
- [3] F. Hauser, M. Häberle, D. Merling, S. Lindner, V. Gurevich, F. Zeiger, R. Frank, and M. Menth, "A survey on data plane programming with p4: Fundamentals, advances, and applied research," *arXiv preprint arXiv:2101.10632*, 2021.
- [4] X. Jin, X. Li, H. Zhang, R. Soulé, J. Lee, N. Foster, C. Kim, and I. Stoica, "Netcache: Balancing key-value stores with fast in-network caching," in *Proceedings of the 26th Symposium on Operating Systems Principles*, pp. 121–136, 2017.
- [5] T. Qu, R. Joshi, M. C. Chan, B. Leong, D. Guo, and Z. Liu, "Sqr: In-network packet loss recovery from link failures for highly reliable datacenter networks," in *2019 IEEE 27th International Conference on Network Protocols (ICNP)*, pp. 1–12, IEEE, 2019.
- [6] A. Afanasyev, J. Shi, B. Zhang, L. Zhang, I. Moiseenko, Y. Yu, W. Shang, Y. Huang, J. P. Abraham, S. DiBenedetto, et al., "Nfd developer's guide," *Dept. Comput. Sci., Univ. California, Los Angeles, Los Angeles, CA, USA, Tech. Rep. NDN-0021*, 2014.
- [7] S. Signorello, R. State, J. François, and O. Festor, "Ndn.p4: Programming information-centric data-planes," in *2016 IEEE NetSoft Conference and Workshops (NetSoft)*, pp. 384–389, 2016.
- [8] R. Miguel, S. Signorello, and F. M. V. Ramos, "Named data networking with programmable switches," in *2018 IEEE 26th International Conference on Network Protocols (ICNP)*, pp. 400–405, 2018.
- [9] O. Karrachou, N. Samaan, and A. Karmouch, "Endn: An enhanced ndn architecture with a p4-programmable data plane," in *Proceedings of the 7th ACM Conference on Information-Centric Networking, ICN '20*, (New York, NY, USA), p. 1–11, Association for Computing Machinery, 2020.
- [10] X. Guo, N. Liu, X. Hou, S. Gao, and H. Zhou, "An efficient ndn routing mechanism design in p4 environment," in *2021 2nd Information Communication Technologies Conference (ICTC)*, pp. 28–33, 2021.
- [11] J. Takemasa, Y. Koizumi, and T. Hasegawa, "Vision: toward 10 tbps ndn forwarding with billion prefixes by programmable switches," in *Proceedings of the 8th ACM Conference on Information-Centric Networking*, pp. 13–19, 2021.
- [12] W. So, A. Narayanan, and D. Oran, "Named data networking on a router: Fast and dos-resistant forwarding with hash tables," in *Architectures for Networking and Communications Systems*, pp. 215–225, 2013.